Automating Cybersecurity Tasks with Python

Cybersecurity automation is an important approach for reducing the time and human effort required to perform repetitive security tasks. Modern organizations generate a large number of security events that must be continuously monitored, analyzed, and investigated.

Python is one of the most widely used programming languages for cybersecurity automation because of its simplicity, flexibility, and large collection of security-related libraries.

Python can be used to:

- **Monitor systems and networks.**
- **Analyze security logs.**
- **Detect suspicious activities.**
- **Monitor file integrity.**
- **Detect unauthorized access attempts.**
- **Automate incident-response tasks.**
- **Generate security alerts.**
- **Produce security reports.**
- **Support vulnerability assessment.**

The general security automation process can be represented as:

Collect → Analyze → Detect → Alert → Recommend → Respond

What is Cybersecurity Automation?

Cybersecurity Automation is the use of software and scripts to automatically perform cybersecurity tasks with no need for manual work.

For example, instead of asking a security administrator to manually examine thousands of login events, a Python program can automatically:

1. Collect login events.
2. Analyze the events.
3. Count failed login attempts.
4. Identify suspicious activity.
5. Generate an alert.
6. Recommend an appropriate action and respond.

Basic workflow

Security Events

↓

Python Script

↓

Data Analysis

↓

Threat Detection

↓

Security Alert

↓

Recommended Action and Respond

Why Use Python in Cybersecurity?

Python has several characteristics that make it useful for cybersecurity.

1. Easy to Learn

Python has a simple and readable syntax, which makes it suitable for students and security professionals.

2. Large Number of Libraries

Python provides many built-in and third-party libraries that can support cybersecurity activities.

Examples include:

os, sys, re, json, hashlib, socket, requests, subprocess, logging, datetime.

3. Automation

Python can automate repetitive security operations.

4. Log Analysis

Python can read, filter, search, and analyze large log files.

5. Security Monitoring

Python can support monitoring of systems, applications, and network-related events in authorized environments.

Cybersecurity Tasks That Can Be Automated

Python can be used to automate many cybersecurity activities.

Security Task	Python Application
Log Analysis	Analyze system and application logs
Threat Detection	Identify suspicious activities
File Integrity Monitoring	Detect unauthorized file changes

Security Task	Python Application
Login Monitoring	Monitor authentication events
Network Monitoring	Monitor authorized network activity
Alert Generation	Generate security alerts
Incident Response	Support automated response procedures
Security Reporting	Generate security reports
Vulnerability Assessment	Support security checking
Backup Verification	Verify backup status and integrity

Writing Custom Python Security Scripts

A **custom security script** is a Python program developed to solve a specific security problem.

For example, a security administrator may want to count failed login attempts.

```
failed_attempts = 0
login_events = [
    "failed",
    "failed",
    "success",
    "failed",
    "failed",
    "failed"
]
```

```
for event in login_events:
    if event == "failed":
        failed_attempts += 1
print("Failed attempts:", failed_attempts)
```

Output

```
Failed attempts: 5
```

The program automatically counts failed authentication attempts instead of requiring manual analysis.

Detecting Brute-Force Attacks

One common cybersecurity application is detecting repeated failed login attempts.

Example:

```
failed_attempts = 0
threshold = 5
login_events = [
    "failed",
    "failed",
    "failed",
    "failed",
    "failed"
]
for event in login_events:
    if event == "failed":
        failed_attempts += 1
```

```
if failed_attempts >= threshold:
    print("ALERT!")
    print("Possible brute-force attack detected.")
else:
    print("Normal activity.")
```

How does the program work?

The program:

1. Reads login events.
2. Counts failed attempts.
3. Compares the number with a predefined threshold.
4. Generates an alert if the threshold is exceeded.

This is a simple example of **rule-based threat detection**.

Monitoring Security Logs

Security logs contain valuable information about activities occurring within a system.

Example:

```
10:20 LOGIN_SUCCESS user1
10:21 LOGIN_FAILED user2
10:22 LOGIN_FAILED user2
10:23 LOGIN_FAILED user2
10:24 LOGIN_FAILED user2
```

Python can read and analyze these events.

Example

```
log_data = """
LOGIN_SUCCESS
LOGIN_FAILED
LOGIN_FAILED
LOGIN_FAILED
LOGIN_FAILED
"""
failed = log_data.count("LOGIN_FAILED")
print("Failed logins:", failed)
if failed >= 4:
    print("Warning: Suspicious login activity detected.")
```

Output

```
Failed logins: 4
Warning: Suspicious login activity detected.
```

File Integrity Monitoring

Python can be used to monitor whether important files have been modified.

A common approach is to calculate a cryptographic hash using **SHA-256**.

Example:

```
import hashlib
data = b"Important system file"
file_hash = hashlib.sha256(data).hexdigest()
print("SHA-256:", file_hash)
```

A change to the file content will normally produce a different hash value.

Security workflow

Original File

↓

Calculate SHA-256

↓

Store Hash

↓

Later Verification

↓

Calculate New Hash

↓

Compare Hashes

↓

Changed?

↓ ↓

Yes No

↓ ↓

Alert Normal

This approach is commonly associated with:

File Integrity Monitoring (FIM)

Python for Network Security Monitoring

Python can also support network-security monitoring in authorized environments.

For example, Python's socket module can retrieve basic information about the local system.

Example:

```
import socket
hostname = socket.gethostname()
ip_address = socket.gethostbyname(hostname)
print("Hostname:", hostname)
print("IP Address:", ip_address)
```

Example output:

```
Hostname: Security-PC
IP Address: 192.168.1.10
```

This type of information can be incorporated into larger monitoring systems.

Threat Detection Using Python

Python can implement simple rules for identifying suspicious activities.

Example:

```
events = [
    {"user": "Ali", "failed": 1},
    {"user": "Sara", "failed": 2},
    {"user": "Admin", "failed": 8}
]
for event in events:
    if event["failed"] >= 5:
        print("Suspicious activity:", event["user"])
```

Output

Suspicious activity: Admin

The example demonstrates how security events can be classified according to predefined rules.

Security Alert Generation

After detecting a potential threat, a security system can generate an alert.

Example:

```
risk_level = "HIGH"
if risk_level == "HIGH":
    print("SECURITY ALERT")
    print("Immediate investigation is recommended.")
```

In a real security environment, alerts may be integrated with:

- Security Operations Centers (SOC)
- SIEM platforms
- Email systems
- Security dashboards
- Incident-management systems